

Java Backend Architecture

Interview Series

Chapter 16.2

Dockerfile & Image Best Practices

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 16.2 – Dockerfile & Image Best Practices

Dockerfile Best Practices

Writing efficient Dockerfiles directly impacts image size, build speed, security posture, and deployment reliability. This chapter covers every Dockerfile instruction with production best practices for Java/Spring Boot applications.

Dockerfile Instruction Reference

FROM	Base image. Use minimal: eclipse-temurin:21-jre-jammy (~200MB), distroless/java21 (~90MB), or Alpine-based (~70MB). Always pin to specific digest for reproducibility.
RUN	Execute commands. Combine with && to reduce layers. Use --mount=type=cache with BuildKit for package managers. Each RUN creates a new layer.
COPY	Copy files from build context or another stage. Preferred over ADD. Use specific paths, not COPY . . Use --chown=nonroot:nonroot.
ADD	Like COPY but also handles URLs and auto-extracts tar archives. Only use ADD for tar extraction; use COPY for everything else.
CMD	Default command/arguments. Overridden by docker run arguments. Use exec form ["cmd","arg"] to make process PID 1.
ENTRYPOINT	The executable that always runs. CMD provides default args. Use exec form. --entrypoint flag required to override at runtime.
ENV	Sets environment variables at build AND runtime. Visible in docker inspect. Never put secrets here.
ARG	Build-time variables only (not available at runtime). Pass via --build-arg. Avoid secrets in ARG — they appear in build history.
EXPOSE	Documents intended container port. Does NOT publish the port. Informational only — used by docker run -P and tooling.
USER	Sets the user for subsequent RUN/CMD/ENTRYPOINT. Create non-root user: RUN useradd -r -u 1001 appuser; USER appuser.
WORKDIR	Sets working directory. Creates it if not exists. Use absolute paths. Sets context for RUN, CMD, ENTRYPOINT, COPY, ADD.
HEALTHCHECK	Defines how Docker tests container health. HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health exit 1
LABEL	Metadata key-value pairs. Use for versioning, maintainer, description. Follows OCI image annotations spec.
VOLUME	Creates a mount point. Forces data to be stored outside the container layer. Any writes go to a Docker-managed volume.
ONBUILD	Triggers a command when this image is used as a base for another build. Useful for base image authors.

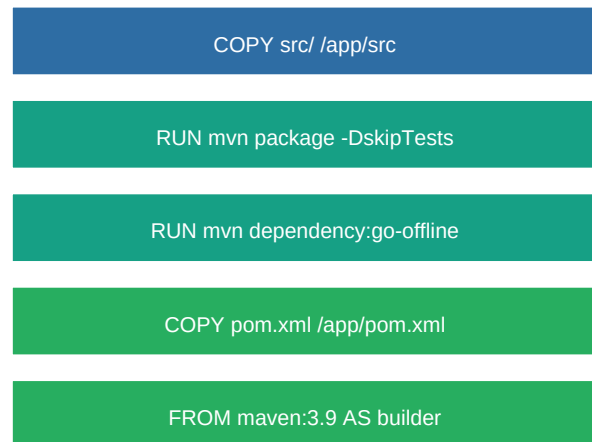
Layer Caching Optimization

Bad Order (cache busted often)



Any src change = full mvn re-run

Good Order (cache-friendly)



src changes only rebuild last layer

Cache busting occurs when a layer changes, invalidating all subsequent layers. Order instructions from least to most frequently changing: base image, dependencies, then source code.

Spring Boot Optimized Dockerfile

```
# Stage 1: Build FROM maven:3.9-eclipse-temurin-21 AS builder WORKDIR /app COPY pom.xml . RUN
mvn dependency:go-offline -B COPY src ./src RUN mvn package -DskipTests -B RUN java
-Djarmode=layertools -jar target/*.jar extract # Stage 2: Runtime FROM
eclipse-temurin:21-jre-jammy RUN adduser --system --uid 1001 appuser WORKDIR /app COPY
--from=builder --chown=appuser:appuser /app/dependencies/ ./ COPY --from=builder
--chown=appuser:appuser /app/spring-boot-loader/ ./ COPY --from=builder
--chown=appuser:appuser /app/snapshot-dependencies/ ./ COPY --from=builder
--chown=appuser:appuser /app/application/ ./ USER appuser EXPOSE 8080 HEALTHCHECK
--interval=30s --timeout=10s --start-period=60s --retries=3 \ CMD curl -f
http://localhost:8080/actuator/health || exit 1 ENTRYPOINT ["java",
"org.springframework.boot.loader.launch.JarLauncher"]
```

CMD vs ENTRYPOINT PID 1 Behavior

CMD vs ENTRYPOINT PID 1 Behavior

```
ENTRYPOINT ["java"] + CMD ["-jar", "app.jar"]
docker run img => java -jar app.jar (PID 1)
```

```
ENTRYPOINT ["java"] + CMD ["-jar", "app.jar"]
docker run img -Xmx512m -jar new.jar => java -Xmx512m -jar new.jar (CMD overridden)
```

```
CMD ["java", "-jar", "app.jar"]
docker run img sh => sh (replaces entire CMD)
```

.dockerignore Impact



Build context size comparison

```
# .dockerignore target/ .git/ .gitignore **/*.log **/*.md node_modules/ .env .env.*
test-results/ *.iml .idea/ .mvn/ !.mvn/wrapper/maven-wrapper.jar
```

Base Image Selection Guide

Base Image	Size	Shell	Package Mgr	Use Case
eclipse-temurin:21-jre-jammy	~200MB	bash	apt	General production
eclipse-temurin:21-jre-alpine	~100MB	sh	apk	Size-sensitive
gcr.io/distroless/java21	~90MB	None	None	Maximum security
gcr.io/distroless/java21:debug	~110MB	busybox	None	Debug distroless
amazoncorretto:21-alpine	~130MB	sh	apk	AWS-optimized
ibm-semeru-runtimes:open-21-jre	~200MB	bash	apt	OpenJ9 JVM

ARG vs ENV Comparison

Feature	ARG	ENV
Availability	Build time only	Build + runtime
Override	--build-arg flag	docker run -e flag
In image history	Yes (avoid secrets!)	Yes (avoid secrets!)
Use case	Build config, version pins	App config, feature flags
Default value	Supported	Supported
Scope	Single build stage	Inherits to all stages

Interview Q&A; – Dockerfile & Image Best Practices

Q1. Why should you combine RUN commands with && in a Dockerfile?

Each RUN instruction creates a new layer in the image. Chaining commands with && in a single RUN reduces the number of layers, reducing image size. More importantly, if you install packages in one RUN and remove them in a later RUN, the packages still exist in the earlier layer — the final image is not smaller. Example: RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/* — the cleanup happens in the same layer so the cache is not stored.

Q2. What is the difference between COPY and ADD in a Dockerfile?

COPY simply copies files from the build context or a previous stage to the container filesystem. ADD does the same but also: (1) Handles URLs (downloads from HTTP URLs). (2) Auto-extracts local tar archives (gzip, bzip2, xz). Best practice: always use COPY unless you specifically need tar extraction or URL download. ADD's auto-extraction behavior can be surprising and lead to unintended content being copied. Docker documentation explicitly recommends COPY over ADD for clarity.

Q3. Explain Docker layer caching and how to optimize it.

Docker caches layers during builds. If a layer and all its parent layers are unchanged, Docker reuses the cached layer. Cache is invalidated when: the instruction changes, the files being COPYed change, or any parent layer is invalidated. Optimization: order instructions from stable (rarely changes) to volatile (frequently changes). For Java: copy pom.xml first, run mvn dependency:go-offline (caches deps layer), then copy source. This way, source changes don't trigger dependency re-download.

Q4. What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the executable that always runs (cannot be overridden without --entrypoint flag). CMD provides default arguments to ENTRYPOINT, or the default command if no ENTRYPOINT is set. Use exec form for both: ENTRYPOINT ["java", "-jar", "app.jar"]. Shell form (ENTRYPOINT java -jar app.jar) spawns a shell as PID 1, which does not forward signals properly. With both defined, docker run img --spring.profiles.active=prod passes --spring.profiles.active=prod as CMD, overriding the default CMD args.

Q5. What is the difference between ENV and ARG?

ARG variables are only available during the build process (at build time). They are set via --build-arg and are NOT available in the running container. ENV variables are available both during build and in the running container. Both appear in docker history — avoid storing secrets in either. Use ARG for build-time config (e.g. APP_VERSION=1.0). Use ENV for runtime config (e.g. SPRING_PROFILES_ACTIVE=prod). Note: ARG before FROM is available to FROM; ARG after FROM requires redeclaration.

Q6. Why is USER nonroot important in a Dockerfile?

Running containers as root means that if the container is compromised, the attacker has root privileges within the container — and potentially on the host if security boundaries are weak. Create a dedicated user: RUN useradd --system --uid 1001 --no-create-home appuser and USER appuser. This follows the principle of least privilege. Note: you must set correct file ownership with COPY --chown=appuser:appuser or chown in RUN before switching to the non-root user. Kubernetes PodSecurityStandard restricted profile requires non-root containers.

Q7. What is .dockerignore and why is it critical?

.dockerignore is a file that specifies patterns of files/directories to exclude from the build context sent to the Docker daemon. Without it, the entire directory (including node_modules, target/, .git/, .env files) is sent to the daemon before building. This causes: slow builds due to large context transfer, potential secrets leakage (if .env or credentials files exist), and incorrect COPY . behavior. Always create .dockerignore excluding: target/,

.git/, node_modules/, *.log, .env files, test data.

Q8. How do you implement HEALTHCHECK in a Dockerfile?

HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health || exit 1 with options: --interval=30s (check frequency), --timeout=10s (command timeout), --start-period=60s (grace period for JVM startup), --retries=3 (failures before unhealthy). Spring Boot Actuator exposes /actuator/health automatically. Healthcheck status appears in docker inspect under State.Health. Docker Swarm and Compose use healthcheck status for depends_on condition: service_healthy. For distroless images (no curl), use a Java-based health check or wget.

Q9. What are distroless images and when should you use them?

Distroless images (from Google) contain only the application and its runtime dependencies — no shell, no package manager, no standard Linux utilities. gcr.io/distroless/java21 contains only the JRE and nothing else (~90MB). Benefits: minimal attack surface (no shell for attacker to use), smaller image, fewer CVEs to patch. Drawbacks: harder to debug (no exec shell). Use the :debug variant for debugging (adds busybox). Ideal for production microservices where security is paramount.

Q10. Explain Spring Boot layered JARs and their benefit for Docker.

Spring Boot 2.3+ supports layered JARs (java -Djarmode=layertools -jar app.jar extract). The JAR is split into 4 layers: dependencies (third-party libs, rarely changes), spring-boot-loader, snapshot-dependencies, application (your code, changes every build). Each layer becomes a separate Docker image layer. Only the changed layer needs to be re-pulled during deployment, dramatically reducing update bandwidth from 200MB to 5-10MB for typical deployments.

Q11. What is the EXPOSE instruction and what does it actually do?

EXPOSE documents that the container listens on the specified port at runtime. It does NOT actually publish the port or create any firewall rule. It serves two purposes: (1) Documentation for developers and operators about which ports the application uses. (2) Used by docker run -P (uppercase P) which automatically maps all EXPOSEd ports to ephemeral host ports. To actually publish a port you must use -p 8080:8080 in docker run or the ports section in docker-compose.yml. EXPOSE is informational.

Q12. How do you pass build-time secrets securely in a Dockerfile?

Never use ENV or ARG for secrets — they appear in docker history and image layers. Use BuildKit secret mounts: RUN --mount=type=secret,id=maven_settings,dst=/root/.m2/settings.xml mvn package. Pass secret at build time: docker build --secret id=maven_settings,src=settings.xml. The secret file is available only during that RUN step and is NOT stored in any image layer. Alternative: use a credential helper that fetches secrets at build time from Vault or AWS Secrets Manager.

Q13. What are multi-platform builds and how do you create them?

Multi-platform builds create images for multiple CPU architectures (linux/amd64, linux/arm64). Use Docker Buildx: docker buildx create --use --name multiplatform and docker buildx build --platform linux/amd64,linux/arm64 -t myimage:latest --push. BuildKit uses QEMU for cross-compilation or connects to native builder nodes. The result is an OCI image index pointing to platform-specific manifests. AWS Graviton (arm64) and Apple Silicon require arm64 images. Modern CI/CD should build multi-platform by default.

Q14. What is the difference between docker build and docker buildx build?

docker build uses the legacy builder (deprecated since Docker 23). docker buildx build uses BuildKit with advanced features: multi-platform builds, cache mounts (--mount=type=cache), secret mounts (--mount=type=secret), SSH mounts, parallel stage execution, better cache management (--cache-from, --cache-to), and inline cache export. BuildKit is enabled by default since Docker 23 (DOCKER_BUILDKIT=1).

for older versions). Always use `docker buildx build` in new projects.

Q15. How do you reduce Docker image size for a Java application?

Key techniques: (1) Use JRE not JDK in final stage (~150MB savings). (2) Multi-stage builds to exclude build tools. (3) Use distroless or Alpine JRE base. (4) Spring Boot layered JARs for efficient layers. (5) JLink to create custom minimal JRE (java.base + required modules only = 50-100MB). (6) Use `.dockerignore` to exclude unnecessary files. (7) Combine RUN commands to reduce layers. (8) Use `--no-install-recommends` with `apt-get`. Typical result: 600MB JDK dev image to 90MB distroless production image.

Q16. Explain BuildKit cache mounts for Maven.

BuildKit cache mounts persist a directory between builds without creating image layers. For Maven: `RUN --mount=type=cache,target=/root/.m2,sharing=locked mvn package`. The `/root/.m2` directory is cached between builds on the same host — Maven only downloads new/changed dependencies. The `sharing=locked` mode prevents concurrent builds from corrupting the cache. This eliminates the slow `mvn dependency:go-offline` pattern needed for layer cache — Maven downloads happen in the cache mount, not in an image layer.

Q17. What LABEL annotations should you add to production images?

Follow OCI image annotation spec (`org.opencontainers.image.*`): `org.opencontainers.image.source` (Git repo URL), `.revision` (Git commit SHA), `.version` (semantic version), `.created` (build timestamp), `.title` (image name), `.description` (what it does), `.licenses` (SPDX license). Add in a single LABEL instruction to avoid extra layers. These labels are queryable via `docker inspect` and are used by security scanners, registries (Harbor), and GitOps tools to trace images back to their source code.

Q18. What is ONBUILD and when would you use it?

ONBUILD registers instructions that execute when the image is used as a base by another Dockerfile. Example: a company base image with `ONBUILD COPY pom.xml /app/` and `ONBUILD RUN mvn dependency:go-offline`. All Dockerfiles FROM that base automatically get these build steps without explicitly writing them. Use case: organizational base images that enforce standard patterns (health checks, non-root user, dependency caching). Drawback: ONBUILD instructions are hidden and can surprise developers — document them clearly.

Q19. How do you implement non-root user with proper file permissions in Docker?

Pattern: (1) Create user in Dockerfile: `RUN useradd --system --uid 1001 --no-create-home --shell /sbin/nologin appuser` (2) Set ownership during COPY: `COPY --from=builder --chown=appuser:appuser /app/app.jar /app/app.jar` (3) Or set ownership with RUN: `RUN chown -R appuser:appuser /app` (4) Switch user: `USER appuser`. For read-only filesystem: ensure all files written at runtime (logs, temp) go to explicitly mounted tmpfs or volumes. Kubernetes: use `securityContext.runAsNonRoot: true` and `runAsUser: 1001` to enforce this.

Q20. What is Docker Scout and how does it help with image security?

Docker Scout analyzes image layers and generates a Software Bill of Materials (SBOM), then matches against CVE databases (NVD, GHSA, etc.). Available via `docker scout quickview` or `docker scout cves`. Integrates with `docker build` to show vulnerabilities at build time. Also recommends base image upgrades. Similar tools: Trivy (open source, very comprehensive), Snyk Container (developer-focused), Gype (Anchore). Integrate into CI/CD to fail builds on critical/high CVEs.

Q21. What are the security implications of using latest tag in production?

The latest tag is mutable — it changes when a new image is pushed with that tag. Using latest in production means: (1) Non-deterministic deployments — you don't know exactly which image version is running. (2)

Unexpected behavior after base image updates. (3) Inability to rollback to a specific version. (4) CI/CD pipelines may deploy different versions to staging vs production. Best practices: pin to specific semantic version (myapp:1.2.3) or better, pin to immutable digest (myapp@sha256:abc...). Use latest only in development.

Q22. How does the WORKDIR instruction affect the build?

WORKDIR sets the working directory for subsequent RUN, CMD, ENTRYPOINT, COPY, and ADD instructions. If the directory doesn't exist, it's created automatically. Use absolute paths (e.g. /app) not relative paths. Multiple WORKDIR instructions stack. WORKDIR is preferred over RUN mkdir + cd because it creates fewer layers and is cleaner. Always set WORKDIR before COPY to ensure files go to the correct location. The working directory is also the default directory when you docker exec -it bash into a container.

Q23. Explain the difference between RUN --mount=type=cache and Docker layer caching.

Docker layer caching caches the result of RUN instructions as image layers. If the instruction and context are unchanged, the layer is reused. RUN --mount=type=cache (BuildKit) provides a persistent cache directory that is NOT stored in image layers. This is ideal for package manager caches (Maven .m2, npm node_modules, pip cache) where you want the directory to persist between builds but not bloat the image. Cache mount is faster and more flexible than layer caching for package downloads.

Q24. What is the FROM instruction and how do you choose a base image?

FROM is the first instruction in every Dockerfile (except ARG before FROM). It sets the base image. Selection criteria: (1) Size — prefer JRE over JDK, Alpine/distroless over Ubuntu for production. (2) Security — fewer packages = fewer CVEs. Eclipse Temurin (from Adoptium) is the recommended free OpenJDK distribution. (3) Support — check EOL dates; use latest LTS (Java 21). (4) Platform — multi-platform support for amd64/arm64. (5) Reproducibility — pin to digest to prevent unexpected base image changes.

Q25. How do you implement multi-stage builds with a test stage?

Add a test stage between builder and runtime: FROM builder AS test; RUN mvn test. Use --target builder or --target test to run only up to that stage in CI. If tests fail, the build fails without producing the runtime image. For integration tests: FROM test AS integration-test; RUN mvn verify -Pintegration-tests. The final production stage COPY --from=builder uses the artifact from the build stage, not the test stage. This ensures test dependencies and test code are never in the production image.

Q26. What build arguments are commonly used in enterprise Java Dockerfiles?

Common ARGs: APP_VERSION (semantic version injected by CI), BUILD_DATE (RFC 3339 timestamp), GIT_COMMIT (SHA for traceability), MAVEN_OPTS (-Xmx2g for large builds), ARTIFACTORY_URL (internal artifact repository), JVM_OPTS (JVM tuning for the build JVM). Pass in CI: docker buildx build --build-arg APP_VERSION=\${VERSION} --build-arg GIT_COMMIT=\${GIT_SHA}. Use these to populate OCI labels for traceability back to the exact source commit.